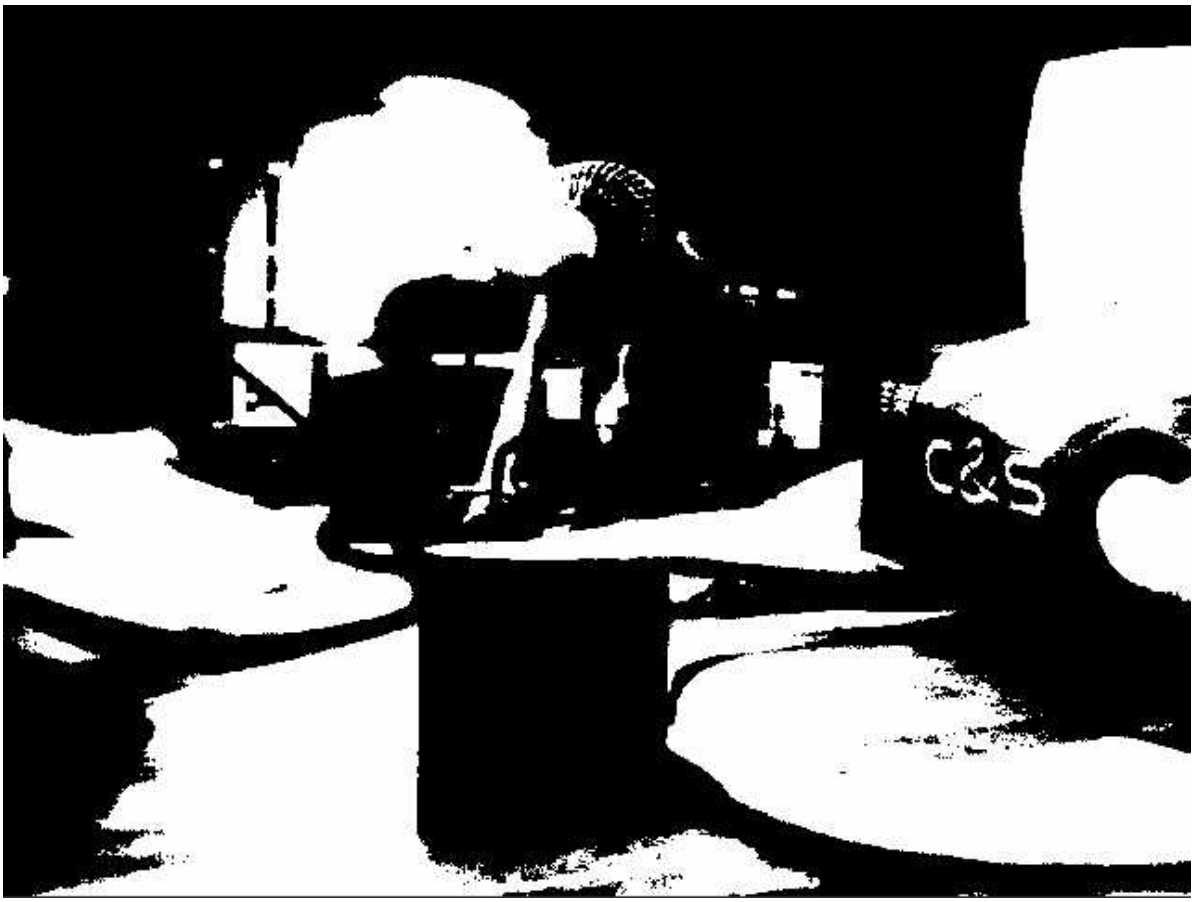
图像二值化-彩色图

图像二值化-彩色图
- 例程效果
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置二值化参数
    - 图像处理与二值化
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/14.rgb888_threshold_binary.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的二值化处理。



代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *      # 摄像头接口 / Camera interface
from media.display import *     # 显示接口 / Display interface
from media.media import *       # 媒体资源管理器 / Media manager
import _thread
import cv_lite                  # cv_lite扩展模块（内含二值化接口）/ cv_lite extension
with binary threshold
import ulab.numpy as np
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=640, height=480)
sensor.reset()
sensor.set_framesize(width=640, height=480)
sensor.set_pixformat(Sensor.RGB888)
```

- 初始化摄像头，设置分辨率为 640x480，帧率未指定（使用默认值）。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的二值化）。

初始化显示模块

```
Display.init(Display.ST7701, width=640, height=480, to_ide=True, quality=50)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=50` 设置图像传输质量。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置二值化参数

```
thresh = 130    # 阈值 / Threshold value
maxval = 255    # 最大值（白）/ Maximum value (white)
clock = time.clock()  # 启动帧率计时器 / Start FPS timer
```

- `thresh`：二值化阈值，小于此值的像素将被设置为 0（黑色），大于等于此值的像素将被设置为 maxval（白色）。
- `maxval`：二值化后的最大像素值，通常设置为 255 表示白色。

- `clock`：用于计算帧率。

图像处理与二值化

```
while True:
    clock.tick()

    # 拍摄一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()

    # 调用二值化接口（返回 ndarray）/ Call binary threshold function (returns
ndarray)
    binary_np = cv_lite.rgb888_threshold_binary(image_shape, img_np, thresh,
maxval)

    # 构造灰度图像用于显示 / Construct grayscale image for display
    img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                          alloc=image.ALLOC_REF, data=binary_np)

    # 显示图像 / Show image
    Display.show_image(img_out)

    # 垃圾回收 & 打印FPS / Garbage collect and print FPS
    gc.collect()
    print("binary:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **二值化处理**：调用 `cv_lite.rgb888_threshold_binary()` 函数进行二值化，返回一个二值化后的灰度图像的 NumPy 数组。
- **图像包装和显示**：将二值化结果包装为灰度图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `thresh`：阈值，值越高需要更亮的像素才能变为白色；根据图像亮度调整，建议从 100 到 200 测试以过滤噪声或突出目标。
- `maxval`：最大值，通常保持为 255；如果需要自定义输出强度，可以调整，但一般无需更改。